

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная №3

Выполнил:

Мокийчук Никита

Группа

J3211

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Мигрировать ранее разработанное приложение (ЛР2) на фреймворк Vue.js. В приложении должен быть подключен роутер, реализована работа с внешним API (желательно посредством axios), выполнено разумное деление на компоненты, а повторяющийся функционал вынесен в composable-функции.

Ход работы

В процессе выполнения лабораторной работы по миграции учебной онлайн-платформы курсов MokiichukKnowledge с JavaScript на Vue.js были выполнены следующие этапы:

- Подготовка окружения и каркаса проекта: был создан новый проект на Vue 3 с использованием Vite в качестве сборщика. Через npm установлены основные зависимости: vue, vue-router для маршрутизации, axios для работы с API, а также bootstrap и bootstrap-icons для оформления интерфейса. Дополнительно подключен json-server, который выполняет роль локального бэкенда и работает с тем же файлом db.json, что и в ЛР2.
- Организация структуры проекта: исходный код распределен по смысловым директориям: api, composables, components, views и router. Такое разделение позволяет понятно отделить сетевой слой от UI-компонентов и переиспользуемой логики, делая структуру проекта более прозрачной для дальнейшей разработки.
- Реализация слоя работы с API через axios: в файле src/api/api.js создан экземпляр axios с базовым адресом `http://localhost:3000` и interceptor-ом, который приводит ошибки к единому виду. На его основе собран объект AppAPI, инкапсулирующий все методы взаимодействия с сервером: получение и создание курсов, уроков, комментариев, регистрация и авторизация, запись на курс, обновление прогресса и редактирование профиля.
- Подключение Vue Router: в файле src/router/index.js описаны маршруты для всех страниц приложения: главной, каталога, страницы курса, входа, регистрации, личного кабинета и кабинета преподавателя. Компоненты страниц подключаются с помощью динамического импорта (lazy-loading). Для приватных страниц добавлен navigation guard, который перенаправляет неавторизованных пользователей на страницу входа, а для маршрута teacher проверяется роль пользователя.

- Деление на компоненты: повторяющиеся фрагменты интерфейса вынесены в отдельные Vue-компоненты: AppNavbar, AppFooter, ToastContainer, CourseCard, ProfileCourseCard, TeacherCourseCard, CommentItem, PasswordInput, EmptyState. Это позволило сделать страницы лаконичными и наглядно продемонстрировать компонентный подход.
- Использование composable-функций: повторяющаяся логика вынесена в директорию src/composables.
- Реактивные формы и работа с данными: формы входа, регистрации, редактирования профиля, добавления курса, урока и комментария реализованы с использованием директивы v-model для двусторонней привязки и обработчика @submit.prevent для отмены стандартной перезагрузки страницы. Каталог курсов фильтруется и сортируется через computed-свойства на основе значений ref-переменных.

Вывод

В ходе выполнения лабораторной работы было выполнено перенос ранее разработанного приложения с JavaScript на фреймворк Vue.js. На практике были изучены и применены ключевые возможности Vue 3: Composition API, реактивность, директивы v-for и v-model, computed-свойства, динамическая маршрутизация через Vue Router и работа с внешним API через axios.

Особое внимание было уделено компонентному подходу и выделению повторяющейся логики в composable-функции, что позволило избежать дублирования кода и сделать архитектуру приложения более масштабируемой. В результате было получено полностью работоспособное приложение, сохранившее весь функционал ЛР2.